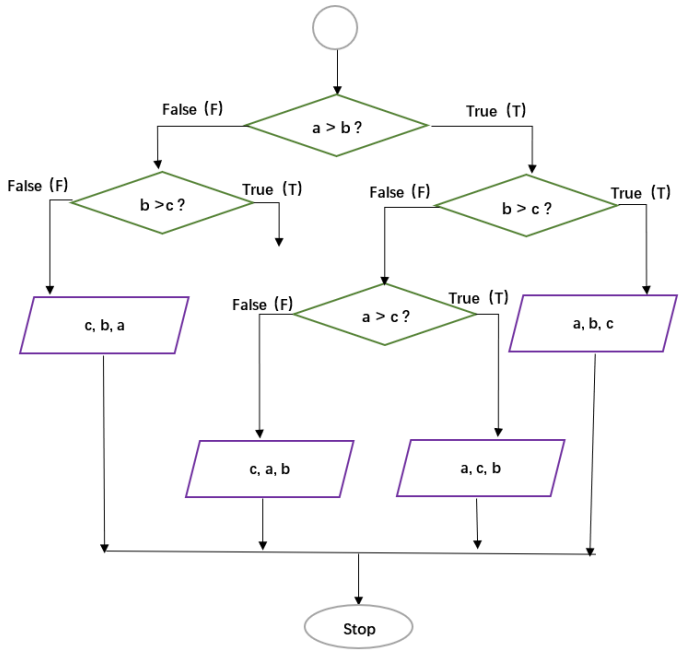
Assignment 1

1. Flowchart

[10 points] Write a function Print_values with arguments a, b, and c to reflect the following flowchart. Here the purple parallelogram operator is to print values in the given order. Report your output with some random a, b, and c values.



The codes are shown below.

```
import random

# get random numbers
a = random.randint(1,10)
b = random.randint(1,10)
c = random.randint(1,10)

#show a,b,c
print(a)
print(b)
print(c)


#logic determine
if a > b :
    if b > c :
        print(a,b,c)

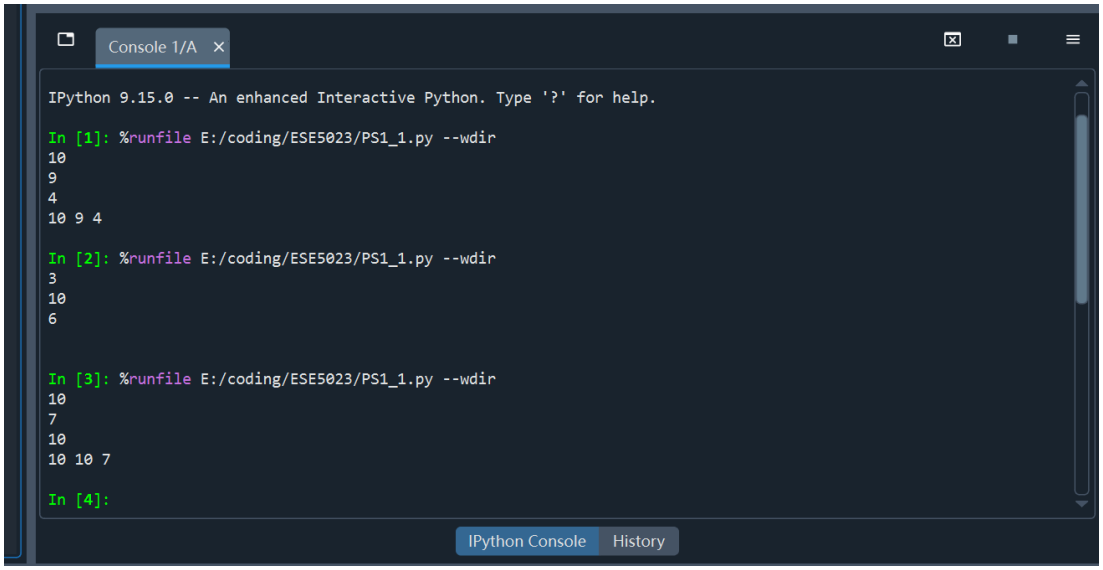
    elif a > c:
        print(a,c,b)

    else:
        print(c,a,b)

elif b > c:
    print(" ")

else:
    print(c,b,a)
```

Here I list 3 results, when a=10,b=9,c=4, the output is a,b,c; when a=3, b=10, c=6, a>b is fulse, then b>c is true, the output is nothing; when a =10, b=7, c=10, a>b is true, b>c is fulse, then the output is a,c,b. The a,b,c are all random numbers from 1-10, the logic is strictly following the flowchart.



2. Matrix multiplication

2.1 [5 points] Make two matrices M1 (5 rows and 10 columns) and M2 (10 rows and 5 columns); both are filled with random integers from 0 and 50.

2.2 [10 points] Write a function Matrix_multip to do matrix multiplication, *i.e.*, M1 * M2. Here you are **ONLY** allowed to use for loop, * operator, and + operator

The codes are shown below:

```python
import random

# 2.1 create the 2 matrixes with randint
M1 = [[random.randint(0, 50) for _ in range(10)] for _ in range(5)]

M2 = [[random.randint(0, 50) for _ in range(5)] for _ in range(10)]

# 2.2 definition of the multiplication of 2 matrixes
def Matrix_multip(A, B):
    # the rows of A
    rows_A = len(A)
    # the columes of A
    cols_A = len(A[0])
    # the columes of B
    cols_B = len(B[0])

    # determine a matrix of the result , get the number of rows of A and columes of B
    result = []
    for i in range(rows_A):
        row = []
        for j in range(cols_B):
            row.append(0)
        result.append(row)

    # multiplication
    #rols of result
    for i in range(rows_A):
        # columes of the result
        for j in range(cols_B):
            # determine the result of each element
            total = 0
            # do the math
            for k in range(cols_A):
                total = total + A[i][k] * B[k][j]
            result[i][j] = total

    return result


# result test
print("M1:")
for row in M1:
    print(row)

print("M2:")
for row in M2:
    print(row)

print("M1 * M2:")
answer = Matrix_multip(M1, M2)
for row in answer:
    print(row)
```

Here I present 2 tested results below:

```
In [4]: %runfile E:/coding/ESE5023/PS1_2.py --wdir
M1:
[32, 24, 22, 41, 13, 47, 22, 23, 42, 15]
[8, 46, 9, 34, 48, 50, 15, 41, 49, 15]
[46, 13, 17, 46, 37, 22, 31, 20, 9, 6]
[8, 19, 13, 49, 1, 42, 48, 15, 12, 29]
[41, 45, 7, 38, 23, 44, 45, 38, 28, 7]
M2:
[21, 8, 17, 7, 43]
[18, 45, 29, 28, 12]
[20, 42, 34, 11, 25]
[45, 39, 14, 13, 42]
[12, 19, 47, 16, 5]
[8, 29, 31, 45, 21]
[30, 1, 18, 7, 45]
[11, 50, 26, 47, 34]
[16, 46, 26, 26, 15]
[3, 26, 46, 3, 10]
M1 * M2:
[5551, 8963, 7406, 6366, 7540]
[5412, 10909, 9358, 8254, 6793]
[5542, 6403, 6390, 4462, 7408]
[5207, 6717, 6064, 4714, 6982]
[6386, 9257, 8065, 7316, 8920]
```

```
In [5]: %runfile E:/coding/ESE5023/PS1_2.py --wdir
M1:
[47, 11, 46, 5, 10, 19, 10, 50, 45, 44]
[0, 47, 19, 9, 5, 26, 36, 36, 44, 35]
[2, 39, 4, 47, 7, 41, 41, 46, 32, 38]
[43, 21, 0, 50, 43, 33, 39, 14, 18, 44]
[1, 8, 33, 32, 34, 16, 16, 20, 46, 26]
M2:
[11, 21, 1, 17, 33]
[47, 28, 37, 49, 1]
[14, 15, 6, 15, 0]
[9, 50, 21, 36, 43]
[20, 28, 11, 41, 22]
[8, 8, 39, 18, 30]
[45, 9, 30, 16, 35]
[42, 39, 44, 26, 4]
[25, 5, 49, 18, 49]
[42, 46, 29, 42, 33]
M1 * M2:
[7598, 6956, 7667, 7078, 6774]
[8566, 5957, 8946, 7359, 6039]
[8975, 8139, 10056, 8746, 7951]
[7675, 8470, 7574, 9077, 9281]
[5747, 5770, 6533, 6434, 6397]
```

3. Pascal triangle

[20 points] One of the most interesting number patterns is Pascal's triangle (named after Blaise Pascal). Write a function Pascal_triangle with an argument k to print the kth line of the Pascal triangle. Report Pascal_triangle(100) and Pascal_triangle(200).

The codes are shown below:

```
def Pascal_triangle(k):

    # the first line: [1]
    line = [1]

    # If k is 0, just print [1]
    if k == 0:
        print(line)
        return

    # Build the line step by step until we reach row k
    # Each new line is built from the previous one
    for i in range(1, k + 1):
        # Create a new list that starts with 1
        new_line = [1]

        # For every pair of neighbours in the old line,
        # add them together to get the middle values
        for j in range(len(line) - 1):
            new_line.append(line[j] + line[j + 1])

        # The new line also ends with 1
        new_line.append(1)

        # The new line becomes the current line for the next round
        line = new_line

    # After the loop, 'line' is the k-th line
    print(line)


# Report the required lines
print("Pascal_triangle(100):")
Pascal_triangle(100)

print("Pascal_triangle(200):")
Pascal_triangle(200)

x = int(input("which line you would like to know in Pascal trangle:"))
Pascal_triangle(x)
```

The results are shown:

```
In [6]: %runfile E:/coding/ESE5023/PS1_3.py --wdir
Pascal_triangle(100):
[1, 100, 4950, 161700, 3921225, 75287520, 1192052400, 16007560800, 186087894300, 1902231808400, 17310309456440,
141629804643600, 1050421051106700, 7110542499799200, 44186942677323600, 253338471349988640, 1345860629046814650,
6650134872937201800, 30664510802988208300, 132341572939212267400, 535983370403809682970, 2041841411062132125600,
7332066885177656269200, 24865270306254660391200, 79776075565900368755100, 242519269720337121015504,
699574816500972464467800, 1917353200780443050763600, 4998813702034726525205100, 12410848711948286545336800,
29372339821610944823963760, 66324638306863423796047200, 143012501349174257560226775, 294692427022540894366527900,
580717429720889409486981450, 1095067153187962886461165020, 1977204582144932989443770175,
3420029547493938143902737600, 5670048986634686922786117600, 9013924030034630492634340800,
13746234145802811501267369720, 20116440213369968050635175200, 28258808871162574166368460400,
38116532895986727945334202400, 49378235797073715747364762200, 61448471214136179596720592960,
73470998190814997343905056800, 84413487283064039501507937600, 93206558875049876949581681100,
98913082887808032681188722800, 100891344545564193334812497256, 98913082887808032681188722800,
93206558875049876949581681100, 84413487283064039501507937600, 73470998190814997343905056800,
61448471214136179596720592960, 49378235797073715747364762200, 38116532895986727945334202400,
28258808871162574166368460400, 20116440213369968050635175200, 13746234145802811501267369720,
9013924030034630492634340800, 5670048986634686922786117600, 3420029547493938143902737600,
1977204582144932989443770175, 1095067153187962886461165020, 580717429720889409486981450,
294692427022540894366527900, 143012501349174257560226775, 66324638306863423796047200, 29372339821610944823963760,
12410848711948286545336800, 4998813702034726525205100, 1917353200780443050763600, 699574816500972464467800,
242519269720337121015504, 79776075565900368755100, 24865270306254660391200, 7332066885177656269200,
2041841411062132125600, 535983370403809682970, 132341572939212267400, 30664510802988208300, 6650134872937201800,
1345860629046814650, 253338471349988640, 44186942677323600, 7110542499799200, 1050421051106700, 141629804643600,
17310309456440, 1902231808400, 186087894300, 16007560800, 1192052400, 75287520, 3921225, 161700, 4950, 100, 1]
Pascal_triangle(200):
```



And here I show some test results of the function:

```
which line you would like to know in Pascal trangle:5
[1, 5, 10, 10, 5, 1]
```

```
which line you would like to know in Pascal trangle:10
[1, 10, 45, 120, 210, 252, 210, 120, 45, 10, 1]
```

```
which line you would like to know in Pascal trangle:30
[1, 30, 435, 4060, 27405, 142506, 593775, 2035800, 5852925, 14307150, 30045015, 54627300, 86493225, 119759850,
145422675, 155117520, 145422675, 119759850, 86493225, 54627300, 30045015, 14307150, 5852925, 2035800, 593775,
142506, 27405, 4060, 435, 30, 1]
```

All correct!

4. Add or double

[20 points] If you start with 1 RMB and, with each move, you can either double your money or add another 1 RMB, what is the smallest number of moves you have to make to get to exactly x RMB? Here x is an integer randomly selected from 1 to 100. Write a function Least_moves to print your results. For example, Least_moves(2) should print 1, and Least_moves(5) should print 3.

The codes are shown below:

```python
#ask the user to give a number to count
x = int(input("get a number from 1-100:"))


def Least_moves(x):


    # Count how many moves we have made
    moves = 0
    "the logic is that we can only +1 or double from 1,"
    "and the least moves from 1 to a specific number is to do the most doubles."
    "so we can devide the aim number with 2, and if can't, -1 until the number get to 1"

    # Work backwards from x down to 1.
    # Going forward:  *2  or  +1
    # Going backward: /2  (only if even)  or  -1
    while x > 1:
        if x % 2 == 0:
            # If x is even, the last forward move could have been *2,
            # so we reverse it by dividing by 2.
            # Dividing is better than subtracting because it reduces
            # the number faster, which keeps the move count minimal.
            x = x // 2
        else:
            # If x is odd, we cannot divide by 2 exactly,
            # so the last forward move must have been +1.
            # We reverse it by subtracting 1.
            x = x - 1

        # One reverse step means one forward move
        moves = moves + 1

    # When x reaches 1, we are back at the start
    print(moves)

#show the least moves
Least_moves(x)
```

The results:

For 2:

```
In [11]: %runfile E:/coding/ESE5023/PS1_4.py --wdir
get a number from 1-100:2
1
```

For 5:

```
In [12]: %runfile E:/coding/ESE5023/PS1_4.py --wdir
get a number from 1-100:5
3
```

For any other numbers:

13

```
In [14]: %runfile E:/coding/ESE5023/PS1_4.py --wdir
get a number from 1-100:13
5
```

100

```
In [15]: %runfile E:/coding/ESE5023/PS1_4.py --wdir
get a number from 1-100:100
8
```

99

```
In [16]: %runfile E:/coding/ESE5023/PS1_4.py --wdir
get a number from 1-100:99
9
```

All correct!

5. Dynamic programming

Insert + or - operation anywhere between the digits 123456789 in a way that the expression evaluates to an integer number. You may join digits together to form a bigger number. However, the digits must stay in the original order.

5.1 [30 points] Write a function Find_expression, which should be able to print every possible solution that makes the expression evaluate to a random integer from 1 to 100. For example, Find_expression(50) should print lines include:

$$1-2+34+5+6+7+8-9=50$$

and

$$1+2+34-56+78-9=50$$

5.2 [5 points] Count the total number of suitable solutions for any integer i from 1 to 100, assign the count to a list called Total_solutions. Plot the list Total_solutions, so which number(s) yields the maximum and minimum of Total_solutions?

The codes are shown:

```
# 5.1

target = int(input("Give 1 number from 1-100:"))

def Find_expression(target):
    digits = "123456789"
    results = []

    # op1 controls gap between 1 and 2, op2 between 2 and 3, ..., op8 between 8 and 9
    for op1 in range(3):
        for op2 in range(3):
            for op3 in range(3):
                for op4 in range(3):
                    for op5 in range(3):
                        for op6 in range(3):
                            for op7 in range(3):
                                for op8 in range(3):

                                    # Store the 8 choices in a list for easy access
                                    ops = [op1, op2, op3, op4, op5, op6, op7, op8]

                                    # build the expression step by step
                                    # 'total'   : sum of all finished terms
                                    # 'current' : the number currently being built
                                    # 'expr'    : the expression string (for printing)

                                    total = 0
                                    current = int(digits[0])       # start with the first digit '1'
                                    expr = "1"

                                    # Walk through each gap and apply the chosen operation
                                    for i in range(8):
                                        # the digit after this gap: 2,3,...,9
                                        d = i + 2
                                        choice = ops[i]
```

```
                                    if choice == 0:
                                        # join: extend the current number
                                        current = current * 10 + d
                                        expr = expr + str(d)
                                    elif choice == 1:
                                        # plus: finish current term, start a new positive term
                                        total = total + current
                                        current = d
                                        expr = expr + "+" + str(d)
                                    else:
                                        # minus: finish current term, start a new negative term
                                        total = total + current
                                        current = -d
                                        expr = expr + "-" + str(d)

                                # After all gaps, the final value is total + current
                                value = total + current

                                # If it matches the target, save the expression
                                if value == target:
                                    results.append(expr)

    # Print all solutions found
    for e in results:
        print(e + "=" + str(target))

    return results

print(Find_expression(target))
```

```
# 5.2


def count_all():
    counts = [0] * 101   # counts[v] = number of expressions equal to v (v = 1..100)

    for op1 in range(3):
        for op2 in range(3):
            for op3 in range(3):
                for op4 in range(3):
                    for op5 in range(3):
                        for op6 in range(3):
                            for op7 in range(3):
                                for op8 in range(3):

                                    ops = [op1, op2, op3, op4, op5, op6, op7, op8]

                                    total = 0
                                    current = 1

                                    # Apply each gap's operation
                                    for i in range(8):
                                        d = i + 2
                                        choice = ops[i]

                                        if choice == 0:
                                            current = current * 10 + d
                                        elif choice == 1:
                                            total = total + current
                                            current = d
                                        else:
                                            total = total + current
                                            current = -d

                                    value = total + current

                                    # Only count values in the range 1..100
                                    if 1 <= value <= 100:
                                        counts[value] = counts[value] + 1
```

```
    return counts


# Run and report

counts = count_all()
Total_solutions = counts[1:101]

print("Total_solutions =", Total_solutions)

maximum = max(Total_solutions)
minimum = min(Total_solutions)

max_numbers = []
min_numbers = []
for i in range(100):
    if Total_solutions[i] == maximum:
        max_numbers.append(i + 1)
    if Total_solutions[i] == minimum:
        min_numbers.append(i + 1)

print("Max count =", maximum, "at number(s):", max_numbers)
print("Min count =", minimum, "at number(s):", min_numbers)


# Plot
import matplotlib.pyplot as plt

numbers = list(range(1, 101))
plt.plot(numbers, Total_solutions, marker="o", markersize=3)
plt.xlabel("Target number (1 to 100)")
plt.ylabel("Number of solutions")
plt.title("Total_solutions for targets 1..100")
plt.grid(True)
plt.show()
```

The results are shown:

5.1:

For 20:

```
In [18]: %runfile E:/coding/ESE5023/PS1_5.py --wdir
Give 1 number from 1-100:20
12+3+4+5+67-89=20
12+3+4+5+6+7-8-9=20
12+3-4+5-6-7+8+9=20
12+3-4-5+6+7-8+9=20
12-3+4+5-6+7-8+9=20
12-3+4-5+6+7+8-9=20
1+23+4+56+7-89=20
1+2+34-5-6-7-8+9=20
1+2-34-56+78+9=20
1-23+4+56-7-8-9=20
1-2+34+5+6-7-8-9=20
['12+3+4+5+67-89', '12+3+4+5+6+7-8-9', '12+3-4+5-6-7+8+9', '12+3-4-5+6+7-8+9', '12-3+4+5-6+7-8+9',
'12-3+4-5+6+7+8-9', '1+23+4+56+7-89', '1+2+34-5-6-7-8+9', '1+2-34-56+78+9', '1-23+4+56-7-8-9',
'1-2+34+5+6-7-8-9']
```

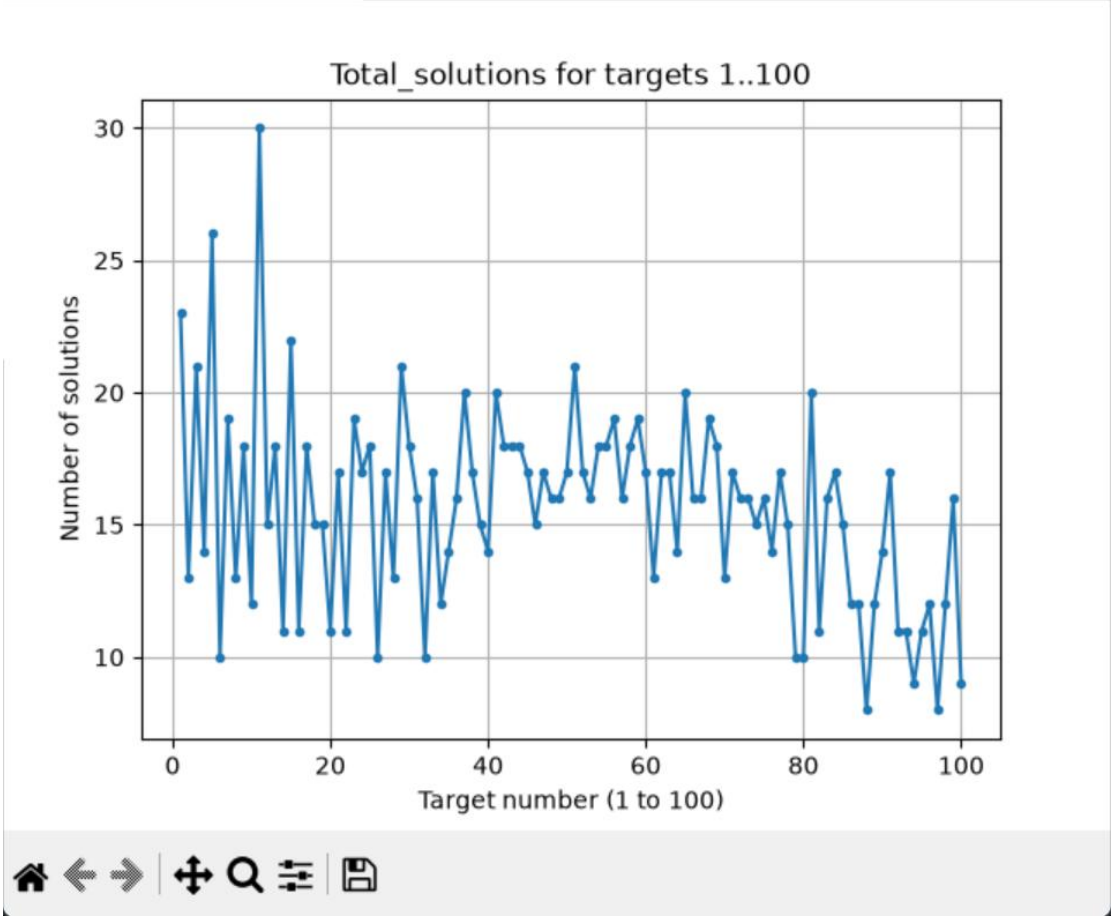
For 60:

```
In [20]: %runfile E:/coding/ESE5023/PS1_5.py --wdir
Give 1 number from 1-100:60
123+4+5+6-7-89=60
123-4-5-67+8-9=60
12+3+45+6-7-8+9=60
12+3+45-6+7+8-9=60
12+3+4+5-67+89=60
12-3-4+5+67-8-9=60
1+23+4+56-7-8-9=60
1+2+34+5-6+7+8+9=60
1+2+3+4+56-7-8+9=60
1+2+3-4-5-6+78-9=60
1+2-34-5+6-7+89=60
1+2-3-4+56+7-8+9=60
1-23-4+56+7+8+9=60
1-23-4+5+6+78-9=60
1-2+3-45+6+78+9=60
1-2+3-4+56+7+8-9=60
1-2-3-4+5-6+78-9=60
['123+4+5+6-7-89', '123-4-5-67+8-9', '12+3+45+6-7-8+9', '12+3+45-6+7+8-9', '12+3+4+5-67+89', '12-3-4+5+67-8-9',
'1+23+4+56-7-8-9', '1+2+34+5-6+7+8+9', '1+2+3+4+56-7-8+9', '1+2+3-4-5-6+78-9', '1+2-34-5+6-7+89',
'1+2-3-4+56+7-8+9', '1-23-4+56+7+8+9', '1-23-4+5+6+78-9', '1-2+3-45+6+78+9', '1-2+3-4+56+7+8-9',
'1-2-3-4+5-6+78-9']
```

And all the other numbers can be count and all correct!

5.2:

```
Total_solutions = [23, 13, 21, 14, 26, 10, 19, 13, 18, 12, 30, 15, 18, 11, 22, 11, 18, 15, 15, 11, 17, 11, 19,
17, 18, 10, 17, 13, 21, 18, 16, 10, 17, 12, 14, 16, 20, 17, 15, 14, 20, 18, 18, 18, 17, 15, 17, 16, 16, 17, 21,
17, 16, 18, 18, 19, 16, 18, 19, 17, 13, 17, 17, 14, 20, 16, 16, 19, 18, 13, 17, 16, 16, 15, 16, 14, 17, 15, 10,
10, 20, 11, 16, 17, 15, 12, 12, 8, 12, 14, 17, 11, 11, 9, 11, 12, 8, 12, 16, 9]
Max count = 30 at number(s): [11]
Min count = 8 at number(s): [88, 97]
```



The results are consistant!